PROBLEM 2 IN DETAIL

Every Iteration Brings x at a New Address

A normal PyTorch loop

```
model = torch.compile(model,
    mode="reduce-overhead")

for i in range(200):
    x = next_batch().to("cuda")          (a)
    y = model(x)                         (b)
    print(y.argmax())                    (c)
```

(a) new input on the GPU, at a new address

(b) compiled model: replay the CUDA graph

(c) ordinary Python on the CPU

The loop still runs on the CPU. The graph only replaces the GPU work inside model(x).

The graph reads a fixed address

First calls of model(x): capture
graph gets an input buffer for x at address A



With PI (Parameter Indirection)
Copy no data: write the new address B into the pointer px (8 bytes).

x moves every iteration, but replay reads the address from capture: copy the data, or use PI.

PROBLEM 2 IN DETAIL

What If x Changes Size?

Frozen at capture

1. **Elements per kernel**
   e.g. xnumel = 4096 is stored as a value
2. **Thread grid**
   how many GPU blocks each kernel launches
3. **Buffer sizes**
   every buffer the graph allocates has a fixed size

A bigger x would be only partly processed and would not fit the buffers, so one graph cannot serve a new size.

PyTorch2: one graph per size

call 1	`x [8, 128]`	new size	record graph #1
call 2	`x [8, 128]`	seen before	replay graph #1
call 3	`x [8, 200]`	new size	record graph #2
call 4	`x [8, 128]`	seen before	replay graph #1
call 5	`x [8, 200]`	seen before	replay graph #2

Each new size is recorded once, then replayed. Many sizes mean many recordings, so in practice inputs are padded to a few fixed sizes (e.g. vLLM).

new address: copy or PI | **new size: new graph**

PI handles a new address, not a new size: each input size gets its own CUDA graph.

Which Kernel Arguments Need PI?

A small model

```
class Attention(nn.Module):
  def __init__(self, d=64):
    self.q = nn.Linear(d, d)                    Wq, bq
    self.k = nn.Linear(d, d)                    Wk, bk
    self.v = nn.Linear(d, d)                    Wv, bv
  def forward(self, x):                          input
    q, k, v = self.q(x), self.k(x), self.v(x)
    s = torch.softmax(q @ k.T / 8, dim=-1)
    return s @ v

model = torch.compile(Attention().cuda(),
                      mode="reduce-overhead")
for x in data:
  y = model(x.to("cuda"))
```

The kernels it becomes

k1	addmm(	x	Wq	bq	buf0)	cuBLAS
k2	addmm(	x	Wk	bk	buf1)	cuBLAS
k3	addmm(	x	Wv	bv	buf2)	cuBLAS
k4	mm(	buf0	buf1	buf3)		cuBLAS
k5	softmax(	buf3	buf4)			Triton
k6	mm(	buf4	buf2	y)		cuBLAS

gets PI (pointer to pointer) left unchanged

k1 to k3 read x, so the prelude updates x in each of them. k4 to k6 only use buffers made inside the graph: untouched.

Only the input x gets PI, in the three kernels that read it. Everything else is left as it was.

IMPLEMENTATION · PARAMETER INDIRECTION

Every Argument, One by One

argument	where it comes from	same address every replay?	PI?
`x`	passed in by the caller	No: a new tensor each call	**Yes**
`Wq, Wk, Wv`	weights of the three nn.Linear	Yes: loaded once	**No**
`bq, bk, bv`	biases of the three nn.Linear	Yes: loaded once	**No**
`buf0, buf1, buf2`	q, k, v: created inside the graph	Yes: the graph reuses its memory	**No**
`buf3, buf4`	scores, softmax: created inside	Yes: the graph reuses its memory	**No**
`y`	output, created inside the graph	Yes: the graph reuses its memory	**No**
`sizes`	numbers, not pointers	Stored as values	**No**

Rule: an argument gets PI only if it points to a model input, not to a tensor the model owns.

IMPLEMENTATION · PARAMETER INDIRECTION

How GraCE Finds Them: Follow Where Each Tensor Came From

1 Dynamo

sees the Python

While tracing, Dynamo knows how each tensor was reached:

`x`
passed in

`Wq, bq, Wk, bk, Wv, bv`
inside self.q/k/v, owned by the model

Every weight and bias the model owns is tagged static.

2 FX graph

the IR

Each outside tensor becomes an input node that keeps its tag:

`input: x`	not static
`input: Wq, bq`	static
`input: Wk, bk`	static
`input: Wv, bv`	static

Everything else is created inside the graph.

3 Inductor code generation

where GraCE acts

For each kernel argument, ask:

created inside the graph?	no PI
input tagged static?	no PI
any other input	PI

The list goes to the LLVM pass (Triton kernels) or to the prelude (library kernels).

GraCE does not watch the data change. It follows where each tensor came from.